

C++ - Module 00

Namespaces, classes, member functions, stdio streams, initialization lists, static, const, and some other basic stuff

Summary:
This document contains the exercises of Module 00 from C++ modules.

Version: 8

Contents

I	**Introduction**	**2**
II	**General rules**	**3**
III	**Exercise 00: Megaphone**	**5**
IV	**Exercise 01: My Awesome PhoneBook**	**6**
V	**Exercise 02: The Job Of Your Dreams**	**8**

Chapter I

Introduction

C++ is a general-purpose programming language created by Bjarne Stroustrup as an extension of the C programming language, or "C with Classes" (source: Wikipedia).

The goal of these modules is to introduce you to **Object-Oriented Programming**. This will be the starting point of your C++ journey. Many languages are recommended to learn OOP. We decided to choose C++ since it's derived from your old friend C. Because this is a complex language, and in order to keep things simple, your code will comply with the C++98 standard.

We are aware modern C++ is way different in a lot of aspects. So if you want to become a proficient C++ developer, it's up to you to go further after the 42 Common Core!

You will discover new concepts step-by-step. The exercises will progressively increase in complexity.

Chapter II

General rules

Compiling

- Compile your code with `c++` and the flags `-Wall -Wextra -Werror`
- Your code should still compile if you add the flag `-std=c++98`

Formatting and naming conventions

- The exercise directories will be named this way: `ex00, ex01, ... , exn`
- Name your files, classes, functions, member functions and attributes as required in the guidelines.
- Write class names in **UpperCamelCase** format. Files containing class code will always be named according to the class name. For instance: `ClassName.hpp`/`ClassName.h`, `ClassName.cpp`, or `ClassName.tpp`. Then, if you have a header file containing the definition of a class "BrickWall" standing for a brick wall, its name will be `BrickWall.hpp`.
- Unless specified otherwise, every output messages must be ended by a new-line character and displayed to the standard output.
- *Goodbye Norminette!* No coding style is enforced in the C++ modules. You can follow your favorite one. But keep in mind that a code your peer-evaluators can't understand is a code they can't grade. Do your best to write a clean and readable code.

Allowed/Forbidden

You are not coding in C anymore. Time to C++! Therefore:

- You are allowed to use almost everything from the standard library. Thus, instead of sticking to what you already know, it would be smart to use as much as possible the C++-ish versions of the C functions you are used to.
- However, you can't use any other external library. It means C++11 (and derived forms) and `Boost` libraries are forbidden. The following functions are forbidden too: `*printf()`, `*alloc()` and `free()`. If you use them, your grade will be 0 and that's it.

- Note that unless explicitly stated otherwise, the `using namespace <ns_name>` and `friend` keywords are forbidden. Otherwise, your grade will be -42.
- **You are allowed to use the STL in the Module 08 and 09 only.** That means: no **Containers** (vector/list/map/and so forth) and no **Algorithms** (anything that requires to include the `<algorithm>` header) until then. Otherwise, your grade will be -42.

A few design requirements

- Memory leakage occurs in C++ too. When you allocate memory (by using the `new` keyword), you must avoid **memory leaks**.
- From Module 02 to Module 09, your classes must be designed in the **Orthodox Canonical Form, except when explicitely stated otherwise**.
- Any function implementation put in a header file (except for function templates) means 0 to the exercise.
- You should be able to use each of your headers independently from others. Thus, they must include all the dependencies they need. However, you must avoid the problem of double inclusion by adding **include guards**. Otherwise, your grade will be 0.

Read me

- You can add some additional files if you need to (i.e., to split your code). As these assignments are not verified by a program, feel free to do so as long as you turn in the mandatory files.
- Sometimes, the guidelines of an exercise look short but the examples can show requirements that are not explicitly written in the instructions.
- Read each module completely before starting! Really, do it.
- By Odin, by Thor! Use your brain!!!



You will have to implement a lot of classes. This can seem tedious, unless you're able to script your favorite text editor.



You are given a certain amount of freedom to complete the exercises. However, follow the mandatory rules and don't be lazy. You would miss a lot of useful information! Do not hesitate to read about theoretical concepts.

Chapter III

Exercise 00: Megaphone

	Exercise : 00
Megaphone	
Turn-in directory : *ex*00/	
Files to turn in : `Makefile, megaphone.cpp`	
Forbidden functions : `None`	

Just to make sure that everybody is awake, write a program that behaves as follows:

```
$>./megaphone "shhhhh... I think the students are asleep..."
SHHHHH... I THINK THE STUDENTS ARE ASLEEP...
$>./megaphone Damnit " ! " "Sorry students, I thought this thing was off."
DAMNIT ! SORRY STUDENTS, I THOUGHT THIS THING WAS OFF.
$>./megaphone
* LOUD AND UNBEARABLE FEEDBACK NOISE *
$>
```



Solve the exercises in a C++ manner.

Chapter IV

Exercise 01: My Awesome PhoneBook

	Exercise : 01
My Awesome PhoneBook	
Turn-in directory : $ex01/$	
Files to turn in : `Makefile, *.cpp, *.{h, hpp}`	
Forbidden functions : `None`	

Welcome to the 80s and their unbelievable technology! Write a program that behaves like a crappy awesome phonebook software.

You have to implement two classes:

- **PhoneBook**
  - It has an array of contacts.
  - It can store a maximum of **8 contacts**. If the user tries to add a 9th contact, replace the oldest one by the new one.
  - Please note that dynamic allocation is forbidden.
- **Contact**
  - Stands for a phonebook contact.

In your code, the phonebook must be instantiated as an instance of the **PhoneBook** class. Same thing for the contacts. Each one of them must be instantiated as an instance of the **Contact** class. You're free to design the classes as you like but keep in mind that anything that will always be used inside a class is private, and that anything that can be used outside a class is public.



`Don't forget to watch the intranet videos.`

On program start-up, the phonebook is empty and the user is prompted to enter one of three commands. The program only accepts ADD, SEARCH and EXIT.

- **ADD**: save a new contact
  - If the user enters this command, they are prompted to input the information of the new contact one field at a time. Once all the fields have been completed, add the contact to the phonebook.
  - The contact fields are: first name, last name, nickname, phone number, and darkest secret. A saved contact can't have empty fields.
- **SEARCH**: display a specific contact
  - Display the saved contacts as a list of **4 columns**: index, first name, last name and nickname.
  - Each column must be **10 characters** wide. A pipe character ('|') separates them. The text must be right-aligned. If the text is longer than the column, it must be truncated and the last displayable character must be replaced by a dot ('.').
  - Then, prompt the user again for the index of the entry to display. If the index is out of range or wrong, define a relevant behavior. Otherwise, display the contact information, one field per line.
- **EXIT**
  - The program quits and the contacts are lost forever!
- **Any other input is discarded.**

Once a command has been correctly executed, the program waits for another one. It stops when the user inputs EXIT.

Give a relevant name to your executable.



http://www.cplusplus.com/reference/string/string/ and of course http://www.cplusplus.com/reference/iomanip/

Chapter V

Exercise 02: The Job Of Your Dreams

	Exercise : 02
The Job Of Your Dreams	
Turn-in directory : *ex*02/	
Files to turn in : `Makefile, Account.cpp, Account.hpp, tests.cpp`	
Forbidden functions : `None`	



`Account.hpp, tests.cpp and the log file are available for download on the intranet page of the module.`

Today is your first day at *GlobalBanksters United*. After successfully passing the recruitment tests (thanks to a few *Microsoft Office* tricks a friend showed you), you joined the dev team. You also know the recruiter was amazed by how quickly you installed *Adobe Reader*. That little extra made all the difference and helped you defeat all your opponents (aka the other applicants): you made it!

Anyway, your manager just gave you some work to do. Your first task is to recreate a lost file. Something went wrong and a source file was deleted by mistake. Unfortunately, your colleagues don't know what `Git` is and use USB keys to share code. At this point, it would make sense to leave this place right now. However, you decide to stay. Challenge accepted!

Your fellow developers give you a bunch of files. Compiling `tests.cpp` reveals that the missing file is `Account.cpp`. Lucky you, the header file `Account.hpp` was saved. There is also a log file. Maybe you could use it in order to understand how the **Account** class was implemented.

You start to recreate the `Account.cpp` file. In only a few minutes, you code a few lines of pure awesome C++. After a couple of failed compilations, your program passes the tests. Its output matches perfectly the one saved in the log file (**except for the timestamps** which will obviously differ since the tests saved in the log file were run before you were hired).

Damn, you're impressive!



The order in which the destructors are called may differ depending on your compiler/operating system. So your destructors may be called in a reverse order.



You can pass this module without doing exercise 02.